Lab Program

1. Missionaries and Cannibals Problem

Aim

To solve the Missionaries and Cannibals problem using Python.

Algorithm

1. Start with 3 missionaries and 3 cannibals on the left side.
2. The boat carries 1 or 2 people.
3. Never allow cannibals to outnumber missionaries where missionaries are present.
4. Move people safely until everyone reaches the right side.

Objective

- To safely move **3 missionaries and 3 cannibals** across a river using a boat.
- Ensure that the number of cannibals never exceeds the number of missionaries on either side (unless no missionaries are present).
- Find the solution using a **State Space Search** algorithm (BFS/DFS).

Flowchart:

```
            Start
              |
              ▼
    Initialize Initial State
              |
              ▼
     Is Goal State Reached?
        ┌─────┴─────┐
       Yes         No
        |           |
        ▼           ▼
   Display    Generate Valid
   Solution      Moves
```

|

▼

Check Safety Rules

|

▼

Update State

|

▼

Goal Reached?

|

No

|

Yes

▼

End

Pseudocode

Start

Initial state = (3,3,1)

Goal state = (0,0,0)

While goal is not reached:

Generate possible boat moves

Check safe state

Add new state

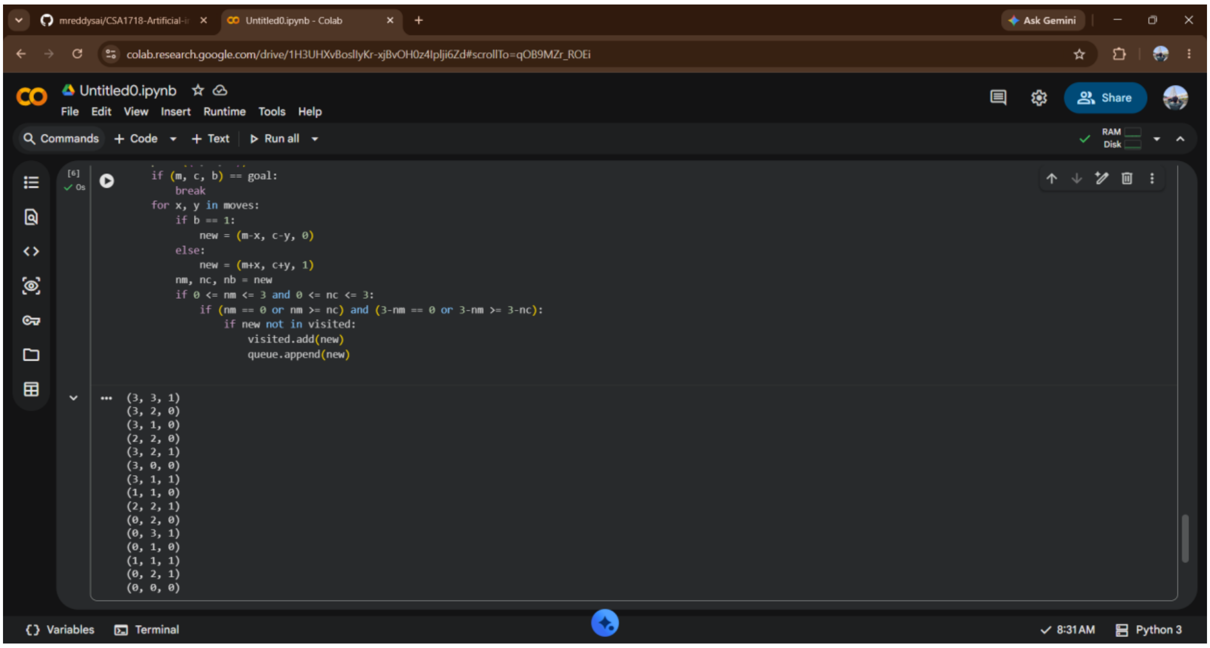
Print solution

Stop

Code

```
from collections import deque
start = (3, 3, 1)
goal = (0, 0, 0)
moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
queue = deque([start])
visited = {start}
while queue:
    m, c, b = queue.popleft()
    print((m, c, b))
    if (m, c, b) == goal:
        break
    for x, y in moves:
        if b == 1:
            new = (m-x, c-y, 0)
        else:
            new = (m+x, c+y, 1)
        nm, nc, nb = new
        if 0 <= nm <= 3 and 0 <= nc <= 3:
            if (nm == 0 or nm >= nc) and (3-nm == 0 or 3-nm >= 3-nc):
                if new not in visited:
                    visited.add(new)
                    queue.append(new)
```

output :



Result:

The Program Missionaries and Cannibals Problem was excueted successfully

2. Vacuum Cleaner Problem

Aim

To solve the Vacuum Cleaner problem using Python.

Algorithm

1. Start with two rooms.
2. Check the current room.
3. If the room is dirty, clean it.
4. Move to the next room.
5. Repeat until all rooms are clean.

Objective

- To clean all dirty rooms using a vacuum cleaner agent.
- The agent checks whether the current room is clean or dirty.

- If dirty, it cleans the room; otherwise, it moves to the next room until all rooms are clean.

Pseudocode

Start

Set rooms as Dirty

For each room:

  If room is Dirty:

    Clean the room

Print final state

Stop

Flowchart:

```
                 Start
                   |
                   ▼
          Place Vacuum in Room
                   |
                   ▼
         Is Current Room Dirty?
              ┌────┴────┐
             Yes       No
              |         |
              ▼         ▼
      Clean Room   Move to Next Room
              |         |
              └────┬────┘
                   ▼
          Are All Rooms Clean?
```

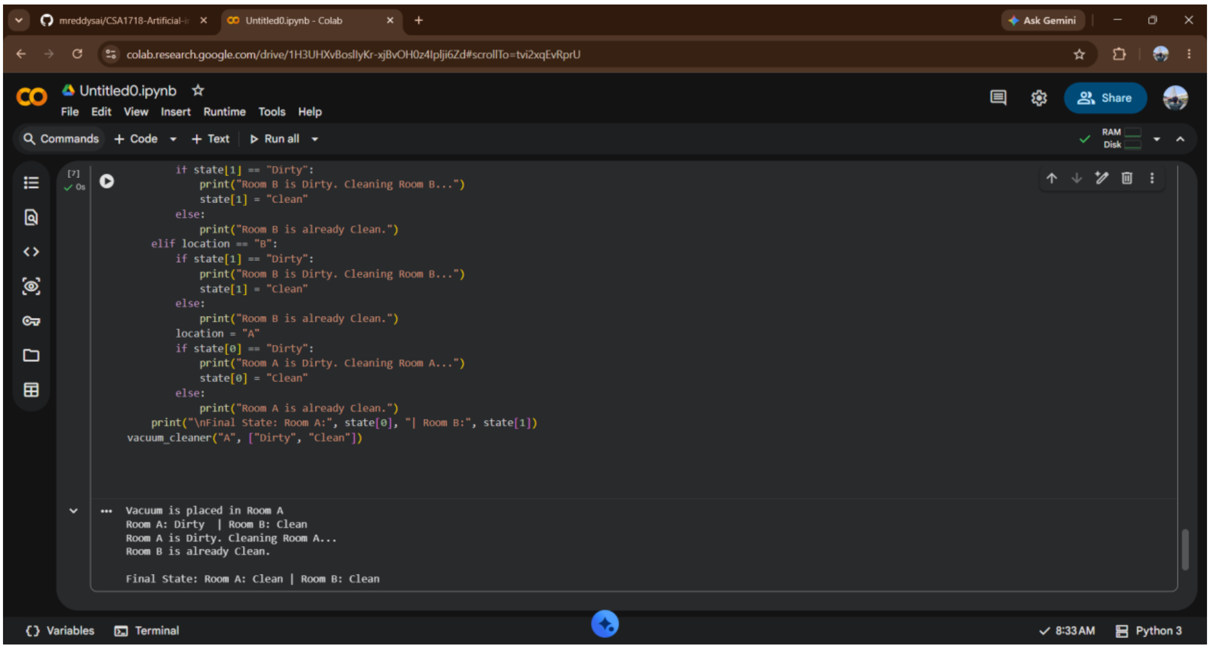
```
┌────┴────┐
Yes       No
 |         |
 ▼         |
Display Success |
 |         |
└────┬────┘
     ▼
    End
```

Code

```python
def vacuum_cleaner(location, state):
    print("Vacuum is placed in Room", location)
    print("Room A:", state[0], " | Room B:", state[1])
    if location == "A":
        if state[0] == "Dirty":
            print("Room A is Dirty. Cleaning Room A...")
            state[0] = "Clean"
        else:
            print("Room A is already Clean.")
        location = "B"
        if state[1] == "Dirty":
            print("Room B is Dirty. Cleaning Room B...")
            state[1] = "Clean"
        else:
            print("Room B is already Clean.")
    elif location == "B":
        if state[1] == "Dirty":
            print("Room B is Dirty. Cleaning Room B...")
            state[1] = "Clean"
        else:
```

```
            print("Room B is already Clean.")
        location = "A"
        if state[0] == "Dirty":
            print("Room A is Dirty. Cleaning Room A...")
            state[0] = "Clean"
        else:
            print("Room A is already Clean.")
    print("\nFinal State: Room A:", state[0], "| Room B:", state[1])
vacuum_cleaner("A", ["Dirty", "Clean"])
```

output



Result:

The Program Vacuum Cleaner Problem was excueted successfully